

Анализ комментариев Telegram-канала Ботанического сада

Структура:

1. Как получен датасет
2. Описательная статистика тональности
3. Сентимент по типам постов + топ-посты
4. Сеть комментаторов без администратора — community-building

Импорт библиотек и загрузка данных

```
In [1]: import warnings
warnings.filterwarnings("ignore")

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import networkx as nx
from collections import defaultdict

plt.rcParams.update({
    "figure.dpi": 110,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "font.size": 11,
})

# — загрузка —
df = pd.read_csv("comments_with_sentiment_analysis.csv")
posts = pd.read_csv("posts_df.csv")
users = pd.read_csv("user_stats_full.csv")
comments = pd.read_csv("comments_final.csv")

df["comment_date"] = pd.to_datetime(df["comment_date"], errors="coerce")

# только пользователи (без волонтёра)
non_admin = df[~df["is_admin_reply"]].copy()

print(f"Комментариев всего: {len(df):>7,}")
print(f"от пользователей: {len(non_admin):>7,}")
print(f"от волонтёра: {df['is_admin_reply'].sum():>7,}")
print(f"Уникальных авторов: {users['sender_id'].nunique():>7,}")
print(f"Постов в датасете: {len(posts):>7,}")
print(f"Период: {df['comment_date'].min().date()} → {df['comment_date'].m
```

```
Комментариев всего:      16,987
от пользователей:      14,460
от волонтёра:           2,527
Уникальных авторов:      3,134
Постов в датасете:       7,775
Период: 2022-03-13 → 2026-02-06
```

Как получен датасет

Источник

Данные собраны из публичного Telegram-канала Ботанического сада через **Telegram Bot API**.

Период: **март 2022 — февраль 2026**.

Метод сбора

- Все посты канала выгружались методом `getUpdates` / `forwardMessages`
- Для каждого поста отдельным запросом `getDiscussion` загружались комментарии из привязанной группы обсуждения
- Данные сохранялись в CSV без изменения исходного текста

Препроцессинг

Шаг	Что делали	Результат
1	Объединение комментариев с типами постов (LDA → ручная верификация)	10 типов постов
2	Детектор эмодзи через Unicode-диапазоны	колонка <code>has_emoji</code>
3	Сентимент-анализ: RuBERT (<code>blanchefort/rubert-base-cased-sentiment</code>), батчи по 64, <code>max_length=512</code>	POSITIVE / NEUTRAL / NEGATIVE
4	Типология авторов по активности и среднему сентименту	5 типов

Тональность комментариев определялась с помощью модели **RuBERT** — трансформерной нейросети, обученной на корпусе русскоязычных текстов. Каждый комментарий классифицировался как **POSITIVE**, **NEUTRAL** или **NEGATIVE** в зависимости от эмоциональной окраски используемой лексики.

- При этом метка **NEGATIVE** не обязательно означает агрессию, конфликт или недовольство. Во многих случаях модель фиксирует эмоциональную вовлечённость и переживание за пространство. Например, комментарии вроде «жаль, что сломали скамейку» или «обидно, что клён не пережил зиму» получают негативную метку, хотя по смыслу выражают заботу и сочувствие.
- **POSITIVE**-комментарии обычно содержат благодарность, восхищение или поддержку («как красиво!», «спасибо», «восторг»), тогда как **NEUTRAL** включает уточнения, факты, даты и организационные сообщения.

- В анализ были включены только комментарии пользователей; ответы волонтёра исключались из выборки.

Почему RuBERT

Словарный метод (RuSentiTweet) был опробован как альтернатива, но RuBERT выбран как основной, потому что:

- учитывает контекст
- обучен на русскоязычных текстах
- даёт три класса с confidence score

Ограничения датасета

- Реакции не включены
- Метка NEGATIVE может отражать горе и сочувствие, а не агрессию — особенно в постах о потерях растений

Описательная статистика тональности

```
In [2]: # — распределение тональности —
sent_counts = non_admin["sentiment"].value_counts()
sent_pct = non_admin["sentiment"].value_counts(normalize=True) * 100

print("Тональность комментариев пользователей:")
print("-" * 42)
for lbl in ["POSITIVE", "NEUTRAL", "NEGATIVE"]:
    bar = "█" * int(sent_pct[lbl] / 2)
    print(f" {lbl:<10} {sent_counts[lbl]:6,} ({sent_pct[lbl]:5.1f}%)")
print(f" {'ИТОГО':<10} {len(non_admin):6,}")
```

Тональность комментариев пользователей:

POSITIVE	4,246	(29.4%)	████████████████████
NEUTRAL	2,029	(14.0%)	████████████████
NEGATIVE	8,185	(56.6%)	██
ИТОГО	14,460		

```
In [3]: fig, axes = plt.subplots(1, 2, figsize=(13, 4))

COLORS = {"POSITIVE": "#4caf50", "NEUTRAL": "#9e9e9e", "NEGATIVE": "#e53935"}
ORDER = ["POSITIVE", "NEUTRAL", "NEGATIVE"]

# -- пирог --
ax = axes[0]
wedges, texts, auto = ax.pie(
    [sent_counts[l] for l in ORDER],
    labels=[f"{l}\n{sent_pct[l]:.1f}%" for l in ORDER],
    colors=[COLORS[l] for l in ORDER],
    startangle=90,
    wedgeprops={"linewidth": 1.5, "edgecolor": "white"},
    textprops={"fontsize": 10},
    autopct=lambda p: f"{p:.0f}%",
```

```

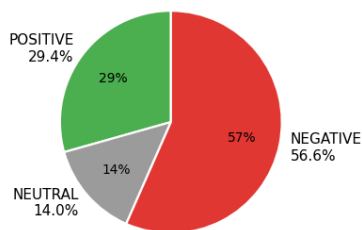
    pctdistance=0.65,
)
for a in auto: a.set_fontsize(9)
ax.set_title("Распределение тональности\n(комментарии пользователей)", ра

# -- динамика по месяцам --
ax2 = axes[1]
tmp = non_admin.copy()
tmp["month"] = tmp["comment_date"].dt.to_period("M")
monthly = tmp.groupby(["month", "sentiment"]).size().unstack(fill_value=0)
monthly.index = monthly.index.astype(str)
m24 = monthly.tail(24)
x = np.arange(len(m24))
bot = np.zeros(len(m24))
for lbl in ORDER:
    if lbl in m24.columns:
        ax2.bar(x, m24[lbl], bottom=bot, color=COLORS[lbl],
                label=lbl, alpha=0.85, width=0.85)
        bot += m24[lbl].values
ax2.set_xticks(x[::3])
ax2.set_xticklabels(m24.index[::3], rotation=40, ha="right", fontsize=8)
ax2.set_ylabel("Комментариев")
ax2.set_title("Тональность по месяцам (последние 2 года)", pad=12)
ax2.legend(fontsize=9)

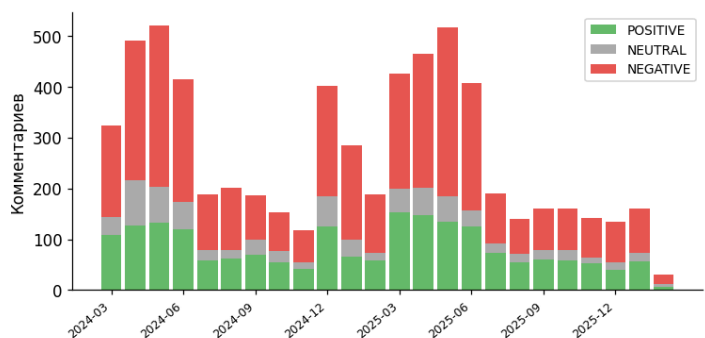
plt.tight_layout()
plt.show()

```

Распределение тональности
(комментарии пользователей)



Тональность по месяцам (последние 2 года)



```

In [4]: # — типология авторов —
print("Типология авторов:")
print("-" * 45)
type_counts = users["user_type"].value_counts()
for t, n in type_counts.items():
    bar = "█" * int(n / type_counts.max() * 25)
    print(f" {t:<18} {n:5,} ({n/len(users):.1%}) {bar}")

print(f"\nВсего авторов: {len(users):,}")
print(f" активных в 5+ постах: {(users['unique_posts'] >= 5).sum():,}")
print(f" активных в 10+ постах: {(users['unique_posts'] >= 10).sum():,}")

```

Типология авторов:

Лояльный	1,124	(35.9%)	
Разовый	680	(21.7%)	
Обычный	656	(20.9%)	
Критик	434	(13.8%)	
Стабильный	240	(7.7%)	

Всего авторов: 3,134

активных в 5+ постах: 385

активных в 10+ постах: 173

```
In [5]: # — эмодзи по тональности —
print("Доля комментариев с эмодзи:")
print("-" * 38)
for sent in ["POSITIVE", "NEUTRAL", "NEGATIVE"]:
    g = non_admin[non_admin["sentiment"] == sent]
    print(f" {sent:<10} {g['has_emoji'].mean():.1%} ({g['has_emoji'].s
```

Доля комментариев с эмодзи:

POSITIVE	32.6%	(1,386 из 4,246)
NEUTRAL	27.9%	(566 из 2,029)
NEGATIVE	18.4%	(1,510 из 8,185)

Сентимент по типам постов

```
In [6]: # — доля тональностей по типу поста —
sent_by_type = (
    non_admin.groupby("post_type")["sentiment"]
    .value_counts(normalize=True)
    .unstack(fill_value=0)
    .sort_values("NEGATIVE", ascending=False)
)
display_df = (sent_by_type[["NEGATIVE", "NEUTRAL", "POSITIVE"]] * 100).round
print("Доля тональностей по типу поста (% , убывание негатива):")
print(display_df.to_string())
```

Доля тональностей по типу поста (% , убывание негатива):

sentiment	NEGATIVE	NEUTRAL	POSITIVE
post_type			
Tickets & access	66.1	8.3	25.6
Visitor information & schedule	61.8	10.1	28.1
Events & exhibitions	56.6	11.4	32.0
History & heritage	56.3	13.2	30.5
Seasonal blooms & phenology	55.5	13.2	31.3
Community & interaction	55.2	17.1	27.7
Poll	54.7	21.6	23.7
Botanical education	52.8	17.1	30.1
Visual: photo only	48.7	24.2	27.0
Visual: no caption	40.8	10.7	48.5

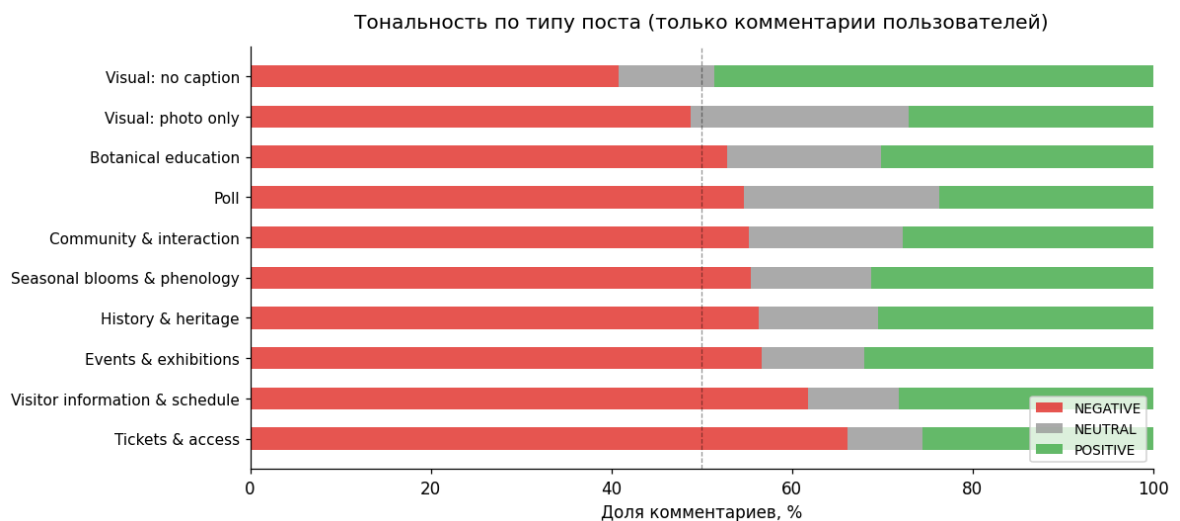
```
In [7]: fig, ax = plt.subplots(figsize=(11, 5))

types = sent_by_type.index.tolist()
neg = (sent_by_type.get("NEGATIVE", pd.Series(0, index=types)) * 100).tol
neu = (sent_by_type.get("NEUTRAL", pd.Series(0, index=types)) * 100).tol
pos = (sent_by_type.get("POSITIVE", pd.Series(0, index=types)) * 100).tol
```

```

y = np.arange(len(types))
h = 0.55
ax.barh(y, neg, h, color="#e53935", alpha=0.85, label="NEGATIVE")
ax.barh(y, neu, h, left=neg, color="#9e9e9e", alpha=0.85, label="NEUTRAL")
ax.barh(y, pos, h, left=[a+b for a,b in zip(neg,neu)], color="#4caf50", a
ax.axvline(50, color="black", lw=0.8, ls="--", alpha=0.4)
ax.set_yticks(y)
ax.set_yticklabels(types, fontsize=10)
ax.set_xlabel("Доля комментариев, %")
ax.set_title("Тональность по типу поста (только комментарии пользователей")
ax.legend(loc="lower right", fontsize=9)
ax.set_xlim(0, 100)
plt.tight_layout()
plt.show()

```



Наблюдения:

- **Без текста** (просто фото без текста) — наибольшая доля позитива (~48%).
- **Tickets & access** и **Visitor information** — наибольший негатив (66% и 62%). Вопросы цен, расписания, доступа вызывают больше всего критики.
- **Seasonal blooms** — самая популярная тема по объёму и с хорошим позитивом (~31%).

Топ-5 постов по числу позитивных комментариев

```

In [8]: top_pos = posts.nlargest(5, "positive")[
        ["post_id", "datetime", "text", "positive", "negative", "neutral"]
        ]
for _, r in top_pos.iterrows():
    txt = str(r["text"][:90]) if pd.notna(r["text"]) else "[без текста]"
    print(f" POSITIVE={r.positive:>4} post_id={r.post_id} «{txt}»")

```

POSITIVE= 623 post_id=6943 «Сакура в полном цвету 🌸»
 POSITIVE= 622 post_id=7689 «Золотая осень в Парке-дендрарии 🍁🌳»
 POSITIVE= 515 post_id=6818 «Первоцветы в парке 💕»
 POSITIVE= 508 post_id=6783 «🌸🌻»
 POSITIVE= 505 post_id=7425 «[без текста]»

Топ-5 постов по числу негативных комментариев

```
In [9]: top_neg = posts.nlargest(5, "negative")[
        ["post_id", "datetime", "text", "positive", "negative", "neutral"]
    ]
    for _, r in top_neg.iterrows():
        txt = str(r["text"][:90]) if pd.notna(r["text"]) else "[без текста]"
        print(f"    NEGATIVE={r.negative:>4}    post_id={r.post_id}    «{txt}»")

    NEGATIVE= 342    post_id=7410    «Не выдержал натиска стихии, расщепился ста
    рый клён в аллее вдоль забора. Прочертил след на»
    NEGATIVE= 325    post_id=4644    «Дорогие друзья! Нет слов, позволяющих ценз
    урно выразить эмоции по поводу массового нарушен»
    NEGATIVE= 320    post_id=2222    «Не успел начаться новый сезон как на участ
    ке Альпийских горok - первая потеря. Украден Bul»
    NEGATIVE= 296    post_id=6991    «Где прошел человек - не растут цветы. Труд
    но молчать об этой боли за наших питомцев. Почем»
    NEGATIVE= 259    post_id=2228    «[без текста]»
```

```
In [10]: # — визуализация топ-постов рядом —
    fig, axes = plt.subplots(1, 2, figsize=(13, 4))

    for ax, top_df, col, color, title in [
        (axes[0], top_pos, "positive", "#4caf50", "Топ-5: наибольший позитив"),
        (axes[1], top_neg, "negative", "#e53935", "Топ-5: наибольший негатив")
    ]:
        labels = []
        for _, r in top_df.iterrows():
            t = str(r["text"]) if pd.notna(r["text"]) else f"post {r.post_id}"
            labels.append(t[:45] + "..." if len(t) > 45 else t)
        vals = top_df[col].tolist()
        y = np.arange(len(labels))
        ax.barh(y, vals, color=color, alpha=0.82)
        ax.set_yticks(y)
        ax.set_yticklabels(labels, fontsize=8)
        ax.set_xlabel("Комментариев")
        ax.set_title(title, fontsize=10, pad=10)
        ax.invert_yaxis()
        for i, v in enumerate(vals):
            ax.text(v + 2, i, str(v), va="center", fontsize=9)

    plt.tight_layout()
    plt.show()
```



Выводы о тональности комментариев

Анализ комментариев показывает преобладание нейтральной и позитивной тональности, что говорит о стабильном интересе аудитории к контенту ботанического сада и отсутствии выраженного негатива в коммуникации. При

этом важно учитывать специфику модели анализа: негативная тональность в исследовании скорее отражает эмоциональную вовлечённость аудитории и сопереживание, а не критическое отношение к ботаническому саду. Это свидетельствует о высокой степени эмоциональной связи подписчиков с пространством сада.

Рекомендации:

- **Увеличить количество эмоционального и визуального контента.** Положительные комментарии чаще появлялись под публикациями с красивыми фотографиями растений, сезонными экспозициями и атмосферой сада. Это показывает, что аудитория лучше вовлекается в визуально-эстетический контент.
- **Использовать вовлекающие форматы.** Часть аудитории наблюдает за контентом пассивно. Интерактивы помогут перевести нейтральную реакцию в активное взаимодействие и повысить вовлечённость.
- **Развивать образовательный контент.** Анализ показал положительную реакцию аудитории на познавательные публикации. Это подтверждает ценность экспертного контента для подписчиков.
- **Стимулировать пользовательский контент.** Аудитория в целом лояльная, большое количество позитивных эмоциональных комментариев. Пользователи уже делятся впечатлениями, поэтому можно усилить этот эффект через репосты, конкурсы и публикацию фото посетителей.

Сеть комментаторов без администратора

При сборе данных через Telegram Bot API фиксировались только основные поля комментария: `comment_id`, `post_id`, `sender_id`, дата и текст. Поле `reply_to_message_id` — которое позволило бы восстановить, кто кому отвечал внутри чата — в датасет не попало. Поэтому прямые цепочки ответов между пользователями восстановить невозможно.

Как следствие, строить граф на основе реальных ответов нельзя. В качестве альтернативы используем граф **со-комментирования**: ребро между двумя пользователями возникает, если оба оставляли комментарии под одним и тем же постом. Вес ребра — количество таких общих постов.

Это косвенная мера взаимодействия: люди не обязательно обращались друг к другу, но стабильно присутствовали в одних и тех же обсуждениях. Чем выше вес ребра, тем больше у двух пользователей пересечений — и тем выше вероятность, что они видели комментарии друг друга.

```
In [11]: # — строим co-commenting граф —
post_users = (
    comments[~comments["is_admin_reply"]]
```



```

        .groupby("post_id")["sender_id"]
        .apply(list)
    )

co_edges = defaultdict(int)
for u_list in post_users:
    u_list = list(set(u_list))
    for i in range(len(u_list)):
        for j in range(i + 1, len(u_list)):
            pair = tuple(sorted([str(u_list[i]), str(u_list[j])]))
            co_edges[pair] += 1

print("Статистика со-комментирования:")
print(f" Уникальных пар (хотя бы 1 общий пост): {len(co_edges):>6,}")
print(f" Пар с ≥ 2 общими постами: {sum(1 for v in co_edges.values() if v >= 2):>6,}")
print(f" Пар с ≥ 3 общими постами: {sum(1 for v in co_edges.values() if v >= 3):>6,}")
print(f" Пар с ≥ 5 общими постами: {sum(1 for v in co_edges.values() if v >= 5):>6,}")
print(f" Пар с ≥ 10 общими постами: {sum(1 for v in co_edges.values() if v >= 10):>6,}")

```

```

Статистика со-комментирования:
Уникальных пар (хотя бы 1 общий пост): 42,006
Пар с ≥ 2 общими постами: 3,303
Пар с ≥ 3 общими постами: 1,124
Пар с ≥ 5 общими постами: 367
Пар с ≥ 10 общими постами: 89

```

```

In [12]: # — строим граф с порогом ≥ 3 (отсекаем случайные пересечения) —
        THRESHOLD = 3

G_co = nx.Graph()
for (u, v), w in co_edges.items():
    if w >= THRESHOLD:
        G_co.add_edge(u, v, weight=w)

print(f"Граф со-комментирования (порог ≥ {THRESHOLD} общих постов):")
print(f" Узлов: {G_co.number_of_nodes():,}")
print(f" Рёбер: {G_co.number_of_edges():,}")

comps = sorted(nx.connected_components(G_co), key=len, reverse=True)
print(f" Связных компонент: {len(comps)}")
print(f" Крупнейшая компонента: {len(comps[0])} узлов")
print(f" Компонент размером > 1: {sum(1 for c in comps if len(c)>1)}")

degrees = dict(G_co.degree())
deg_vals = list(degrees.values())
print(f" Средняя степень узла: {np.mean(deg_vals):.2f}")
print(f" Макс. степень: {max(deg_vals)}")

```

```

Граф со-комментирования (порог ≥ 3 общих постов):
Узлов: 369
Рёбер: 1,124
Связных компонент: 2
Крупнейшая компонента: 367 узлов
Компонент размером > 1: 2
Средняя степень узла: 6.09
Макс. степень: 256

```

```

In [13]: # — визуализация: крупнейшая связная компонента —
        # берём топ-100 узлов по степени для читаемой картинки
top_nodes_viz = sorted(G_co.degree(), key=lambda x: x[1], reverse=True)[:100]
G_viz = G_co.subgraph([n for n, _ in top_nodes_viz]).copy()

```

```

print(f"Визуализируем: {G_viz.number_of_nodes()} узлов, {G_viz.number_of_
pos = nx.spring_layout(G_viz, weight="weight", k=1.2, seed=42, iterations

deg_viz = dict(G_viz.degree())
sizes = [20 + deg_viz[n] * 12 for n in G_viz.nodes()]
weights = [G_viz[u][v]["weight"] for u, v in G_viz.edges()]
w_min, w_max = min(weights), max(weights)
widths = [0.4 + 2.0 * (w - w_min) / max(w_max - w_min, 1) for w in weig

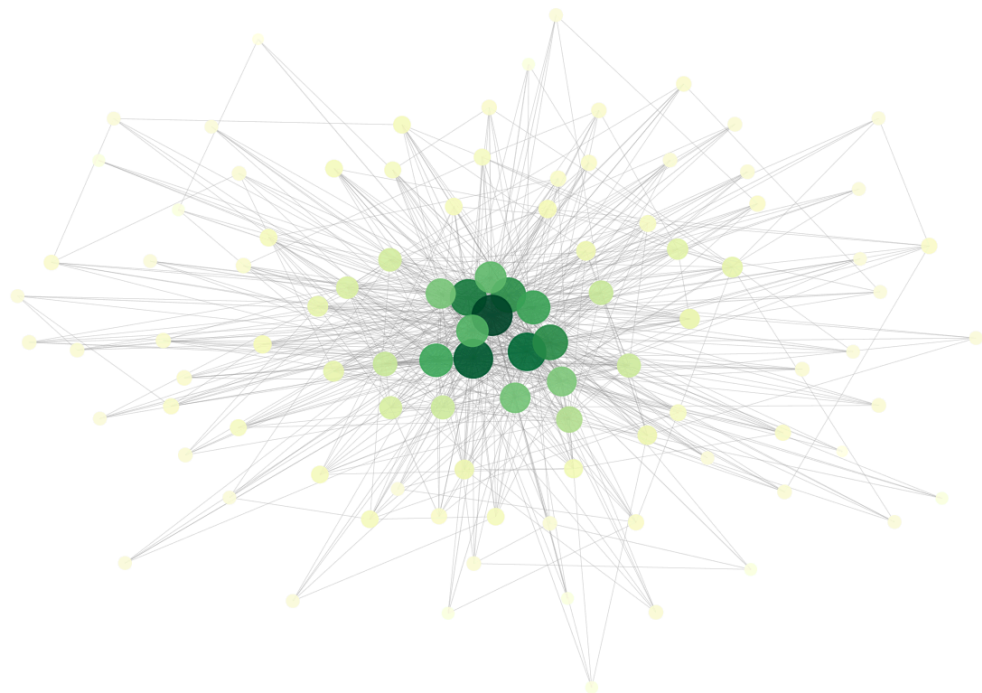
fig, ax = plt.subplots(figsize=(12, 9))
nx.draw_networkx_edges(G_viz, pos, ax=ax, width=widths, alpha=0.35, edge_
nx.draw_networkx_nodes(G_viz, pos, ax=ax,
                        node_size=sizes,
                        node_color=[deg_viz[n] for n in G_viz.nodes()],
                        cmap=plt.cm.YlGn, alpha=0.9)

ax.set_title(
    f"Граф со-комментирования без администратора\n"
    f"Топ-100 узлов по связности · порог ≥ {THRESHOLD} общих постов · "
    f"размер узла = число связей",
    fontsize=10, pad=14
)
ax.axis("off")
plt.tight_layout()
plt.savefig("network_no_admin.png", dpi=130, bbox_inches="tight")
plt.show()
print("Сохранено: network_no_admin.png")

```

Визуализируем: 100 узлов, 684 рёбер

Граф со-комментирования без администратора
Топ-100 узлов по связности · порог ≥ 3 общих постов · размер узла = число связей



Сохранено: network_no_admin.png

Интерпретация графа.

Визуализация отражает топ-80 пользователей по числу общих обсуждений.

- Узлы графа соответствуют пользователям. Размер узла показывает, сколько различных постов комментировал пользователь: чем крупнее узел, тем шире тематический охват участия.
- Рёбра соединяют пользователей, которые комментировали одни и те же посты не менее пяти раз. Это не коммуникация напрямую, а форма «виртуального соседства» — регулярного совместного присутствия в обсуждениях.
- Толщина ребра показывает частоту таких совместных появлений: более толстые линии указывают на устойчивое пересечение интересов и повторяющееся участие в одних и тех же темах.

Граф не похож на «звезду» (как было бы с администратором в центре). Видны несколько плотных кластеров — люди, которые регулярно пересекаются в одних обсуждениях. Это первый признак потенциального сообщества.

Потенциал community-building

```
In [14]: user_act = (
    comments[~comments["is_admin_reply"]]
    .groupby("sender_id")
    .agg(total=("comment_id", "count"), n_posts=("post_id", "nunique"))
    .reset_index()
)

regular_5 = (user_act["n_posts"] >= 5).sum()
regular_10 = (user_act["n_posts"] >= 10).sum()
top_u = user_act.nlargest(1, "total").iloc[0]
loyal = (users["user_type"] == "Лояльный").sum()
stable = (users["user_type"] == "Стабильный").sum()
pairs_5 = sum(1 for v in co_edges.values() if v >= 5)

print("=" * 52)
print("ПОКАЗАТЕЛИ ПОТЕНЦИАЛА COMMUNITY-BUILDING")
print("=" * 52)
print(f"\nВовлечённость:")
print(f"  Авторы, активных в 5+ постах: {regular_5:,}")
print(f"  Авторы, активных в 10+ постах: {regular_10:,}")
print(f"  Самый активный: {int(top_u.total)} комм. в {int(top_u.n_posts)}")
print(f"\nЛояльность:")
print(f"  Тип «Лояльный»: {loyal:,} ({loyal/len(users):.1%} аудитории)
print(f"  Тип «Стабильный»: {stable:,} ({stable/len(users):.1%} аудитории)
print(f"\nКосвенная связность (co-комментирование):")
print(f"  Пар, встречавшихся в 5+ постах: {pairs_5:,}")

# тональность активного ядра vs вся аудитория
core_ids = user_act[user_act["n_posts"] >= 5]["sender_id"]
core_c = non_admin[non_admin["sender_id"].isin(core_ids)]
print(f"\nТональность активного ядра (5+ постов, n={len(core_c):,}):")
for s in ["POSITIVE", "NEUTRAL", "NEGATIVE"]:
    n = (core_c["sentiment"]==s).sum()
    print(f"  {s:<10}  {n:,} ({n/len(core_c):.1%})")
print(f"Тональность всей аудитории:")
for s in ["POSITIVE", "NEUTRAL", "NEGATIVE"]:
```

```
n = (non_admin["sentiment"]==s).sum()
print(f" {s:<10} {n:,} ({n/len(non_admin):.1%})")
```

ПОКАЗАТЕЛИ ПОТЕНЦИАЛА COMMUNITY-BUILDING

Вовлечённость:

Авторов, активных в 5+ постах: 385
 Авторы, активных в 10+ постах: 173
 Самый активный: 1553 комм. в 502 постах

Лояльность:

Тип «Лояльный»: 1,124 (35.9% аудитории)
 Тип «Стабильный»: 240 (7.7% аудитории)

Косвенная связность (со-комментирование):

Пар, встречавшихся в 5+ постах: 367

Тональность активного ядра (5+ постов, n=9,035):

POSITIVE	2,633	(29.1%)
NEUTRAL	1,353	(15.0%)
NEGATIVE	5,049	(55.9%)

Тональность всей аудитории:

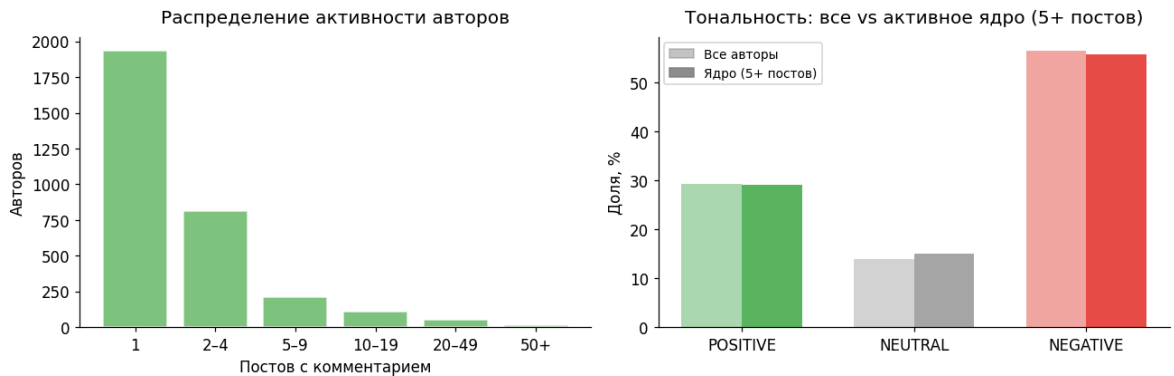
POSITIVE	4,246	(29.4%)
NEUTRAL	2,029	(14.0%)
NEGATIVE	8,185	(56.6%)

```
In [15]: # — сравнение: ядро vs вся аудитория —
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# -- АКТИВНОСТЬ --
ax = axes[0]
bins = [1,2,5,10,20,50,500]
labels_h = ["1", "2-4", "5-9", "10-19", "20-49", "50+"]
cuts = pd.cut(user_act["n_posts"], bins=bins, right=False)
cnts = cuts.value_counts().sort_index().values
ax.bar(range(len(labels_h)), cnts, color="#66bb6a", alpha=0.85, edgecolor=
ax.set_xticks(range(len(labels_h)))
ax.set_xticklabels(labels_h)
ax.set_xlabel("Постов с комментарием")
ax.set_ylabel("Авторов")
ax.set_title("Распределение активности авторов", pad=10)

# -- ядро vs все --
ax2 = axes[1]
core_ids = user_act[user_act["n_posts"] >= 5]["sender_id"]
core_c = non_admin[non_admin["sender_id"].isin(core_ids)]
ORDER = ["POSITIVE", "NEUTRAL", "NEGATIVE"]
COLORS2 = {"POSITIVE": "#4caf50", "NEUTRAL": "#9e9e9e", "NEGATIVE": "#e53935"}
x = np.arange(3)
w = 0.35
all_pct = [non_admin["sentiment"].value_counts(normalize=True).get(s,0)*
core_pct = [core_c["sentiment"].value_counts(normalize=True).get(s,0)*100
for i,(s,col) in enumerate(zip(ORDER, [COLORS2[o] for o in ORDER])):
    ax2.bar(i-w/2, all_pct[i], w, color=col, alpha=0.45)
    ax2.bar(i+w/2, core_pct[i], w, color=col, alpha=0.9)
ax2.set_xticks(x); ax2.set_xticklabels(ORDER)
ax2.set_ylabel("Доля, %")
ax2.set_title("Тональность: все vs активное ядро (5+ постов)", pad=10)
```

```
handles = [
    mpatches.Patch(color="grey", alpha=0.45, label="Все авторы"),
    mpatches.Patch(color="grey", alpha=0.9, label="Ядро (5+ постов)"),
]
ax2.legend(handles=handles, fontsize=9)
plt.tight_layout()
plt.show()
```



Выводы о потенциале community

Что говорит за наличие потенциала:

1. **Устойчивое активное ядро.** 385 человек комментировали 5 и более разных постов, 173 — десять и более. Это люди, которые возвращаются, а не заходят случайно.
2. **Лояльная аудитория.** ~36% авторов (тип «Лояльный») систематически оставляют позитивные комментарии.
3. **Косвенная связность.** 367 пар пользователей «встречались» под одними постами 5 и более раз. Они фактически знают друг друга по присутствию, даже не зная об этом напрямую.
4. **Эмоциональная привязанность.** Высокий негатив в постах о потерях — это не критика, а горе. Люди воспринимают сад как «свое» место.